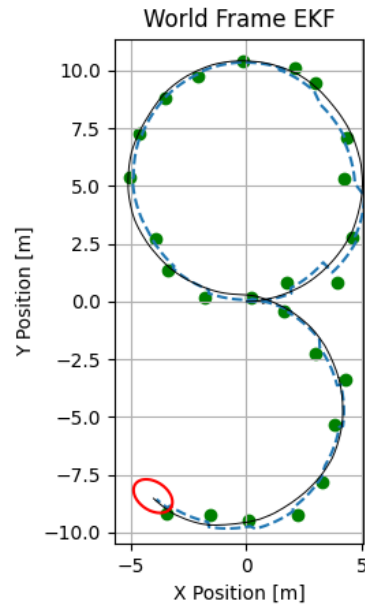


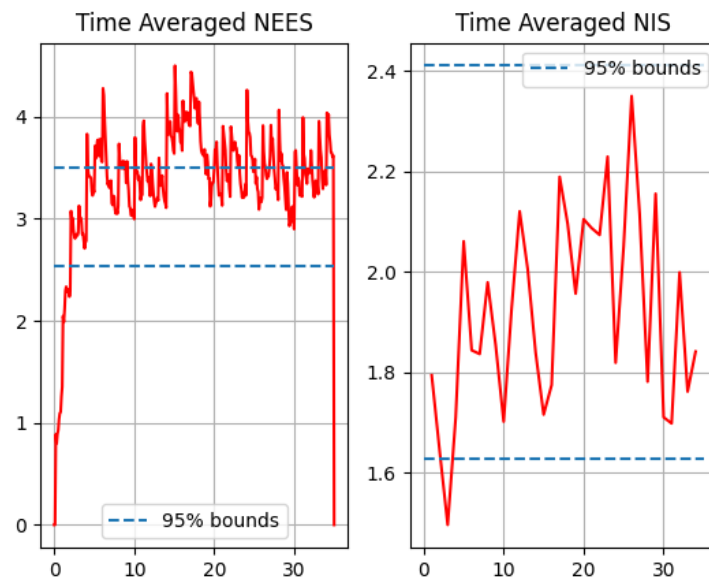
2.d)

Figure 1: Animation of EKF



2.e)

Figure 2: Monte Carlo Evaluation of EKF



Code Snippet 1: EKF Code

```

1 import numpy as np
2 import sympy as sp
3 import matplotlib.pyplot as plt
4 import matplotlib.animation as animation
5 from Code.SE2.SE2_maps import se2_compose, se2_exp, se2_log, se2_wedge, se2_vice, se2_inverse
6 from Code.SE2.SE2_integrators import se2_euler_integrator, se2_Lie_group_integrator
7 from Code.Plotting.covariance_plotting import plot_cov_ellipse, calc_and_plot_cov_ellipse,
8 from Code.Plotting.SE2_plotting import plot_scatter_uncycles,
9 from HW4.Parts.AAE590LGM_HW4_Problem import AAE590LGM_HW4_Problem
10 from Code.Filtering.KalmanFilter import EKF
11 from Code.Filtering.filter_performance_metrics import averaged_chi_squared_consistency_bands
12
13 # Problem 2: World-Frame EKF
14
15 ## Unicycle SE2 Dynamics
16 #  $\dot{X} = X \hat{\xi}$ 
17 #  $\xi = [v; 0; \omega]$ 
18
19 ## Initialize Problem Parameters
20 # Simulation
21 prob = AAE590LGM_HW4_Problem()
22
23 # Monte Carlo
24 N_runs = 100
25
26 def EKF_SE2(prob):
27     t_ref = prob.t_k
28     X_ref = prob.simulate(np.zeros((3, prob.N_nodes)))
29
30     def f_disc(x_k, xi_k, dt):
31         x_kp1 = [x_k[0] + xi_k[0] * sp.cos(x_k[2]) * dt,
32                 x_k[1] + xi_k[0] * sp.sin(x_k[2]) * dt,
33                 x_k[2] + xi_k[2] * dt]
34         return x_kp1
35
36     # 2a) Prediction Step
37     x_sym = sp.symbols("x:3")
38     xi_sym = sp.symbols("xi:3")
39     f_disc_sym = sp.Matrix(f_disc(x_sym, xi_sym, prob.delta_t))
40     F = f_disc_sym.jacobian(x_sym)
41     F_func = sp.lambdify((x_sym, xi_sym), F, "numpy")
42
43     f_disc_func = sp.lambdify((x_sym, xi_sym), f_disc_sym, "numpy")
44
45     L_k = lambda theta : prob.delta_t * np.array([[np.cos(theta), 0, 0], [np.sin(theta), 0, 0], [0, 0, 1]])
46     Q_k_world = lambda x : L_k(x[2]) @ prob.Q_body @ L_k(x[2]).T
47
48     # 2b) Measurement Update
49     h_GPS = lambda x : x[0:2]
50     h_GPS_noise = lambda x : prob.rng.normal(h_GPS(x), prob.sigma_GPS)
51     R_z_GPS = prob.sigma_GPS ** 2 * np.eye(2)
52     H_GPS = np.hstack((np.eye(2), np.zeros((2, 1))))
53     H_GPS_func = lambda x : H_GPS
54
55     # 2c) Implementation
56     xhat_0 = np.zeros((3, 1))
57     P_0 = np.diag((0.01, 0.01, 0.01))
58
59     X_true = prob.monte_carlo(1)
60     x_true = np.zeros((3, prob.N_nodes))
61     for k in range(prob.N_nodes):
62         x_true[:, k] = se2_mat_to_tuplevec(X_true[:, :, k]).reshape((3,))
63

```

```

64     xhat_k = np.zeros((3, prob.N_nodes))
65     Xhat_k = np.zeros((3, 3, prob.N_nodes))
66     Phat_k = np.zeros((3, 3, prob.N_nodes))
67     xhat_k[:, 0] = xhat_0.reshape((3,))
68     Xhat_k[:, :, 0] = se2_tuplevec_to_mat(xhat_0).reshape((3, 3))
69     Phat_k[:, :, 0] = P_0
70     nu_k = []
71     K_k = []
72     innovation = []
73     t_k_GPS = []
74     z_k_GPS = []
75     k_GPS = []
76     for k in range(prob.N_steps):
77         if np.mod(t_ref[k], 1 / prob.GPS_rate) == 0 and k > 1:
78             z_GPS = h_GPS_noise(x_true[:, k])
79             z_k_GPS.append(z_GPS)
80             t_k_GPS.append(t_ref[k])
81             k_GPS.append(k)
82
83             xhat_k[:, k + 1], Phat_k[:, :, k + 1], nu_k_i, K_k_i, innovation_i = EKF(xhat_k[:, k],
84             Phat_k[:, :, k], prob.xi_k[:, k], h_GPS, z_GPS, f_disc_func, F_func, Q_k_world,
85             H_GPS_func, R_z_GPS)
86
87             nu_k.append(nu_k_i)
88             K_k.append(K_k_i)
89             innovation.append(innovation_i)
90         else:
91             z_GPS = np.nan
92
93             xhat_k[:, k + 1], Phat_k[:, :, k + 1] = EKF(xhat_k[:, k], Phat_k[:, :, k], prob.
94             xi_k[:, k], h_GPS, z_GPS, f_disc_func, F_func, Q_k_world, H_GPS_func, R_z_GPS)
95             Xhat_k[:, :, k + 1] = se2_tuplevec_to_mat(xhat_k[:, k + 1]).reshape((3, 3))
96
97             nu_k = np.array(nu_k)
98             K_k = np.array(K_k)
99             innovation = np.array(innovation)
100             t_k_GPS = np.array(t_k_GPS)
101             z_k_GPS = np.array(z_k_GPS)
102             k_GPS = np.array(k_GPS)
103
104     return x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k, K_k, innovation, t_k_GPS, z_k_GPS,
105     k_GPS
106 x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k, K_k, innovation, t_k_GPS, z_k_GPS, k_GPS =
107     EKF_SE2(prob)
108
109 # 2d Animation
110
111 fig, ax = plt.subplots()
112 ax.set_title("World Frame EKF")
113 ax.set_xlabel("X Position [m]")
114 ax.set_ylabel("Y Position [m]")
115 ax.grid()
116 plt.gca().set_aspect('equal', adjustable = 'box')
117 run_plot, run_quiver, ref_plot, ref_quiver = plot_scatter_unicycles_ax(X_true[:, :, 0],
118     Xhat_k[:, :, 0].reshape((3, 3, 1, 1)), True, ax)
119 cov_plot = plot_cov_ellipse_ax(Phat_k[0:2, 0:2, 0], xhat_k[0:2, 0], "red", "-", ax)
120 gps_plot = ax.scatter(z_k_GPS[k_GPS <= prob.N_nodes, 0], z_k_GPS[k_GPS <= prob.N_nodes, 1],
121     color = "green")
122
123 def update(k):
124     ref_plot[0].set_xdata(x_true[0, :k])
125     ref_plot[0].set_ydata(x_true[1, :k])
126     run_plot[0].set_xdata(xhat_k[0, :k])
127     run_plot[0].set_ydata(xhat_k[1, :k])
128     e_data = create_cov_ellipse(Phat_k[0:2, 0:2, k], xhat_k[0:2, k])
129     cov_plot[0].set_xdata(e_data[0, :] + xhat_k[0, k])
130     cov_plot[0].set_ydata(e_data[1, :] + xhat_k[1, k])
131     gps_plot.set_offsets(z_k_GPS[k_GPS <= k, :])

```

```

125     return (gps_plot, cov_plot[0], ref_plot, run_plot)
126
127 ani = animation.FuncAnimation(fig=fig, func=update, frames = prob.N_nodes, interval=10)
128
129 plt.savefig("./HW4/Parts/Q1/AAE590LGM_HW4_Q2_ref.png")
130 plt.show()
131
132 # 2e) Monte Carlo Evaluation
133 nu_k_runs = np.zeros((N_runs, t_k_GPS.size))
134 eps_k_runs = np.zeros((N_runs, prob.N_nodes))
135 for i in range(N_runs):
136     x_true, X_true, xhat_k, Xhat_k, Phat_k, nu_k_runs[i, :], K_k, innovation, t_k_GPS,
137     z_k_GPS, k_GPS = EKF_SE2(prob)
138
139     xhat_error = x_true[:, 0:] - xhat_k[:, 0:]
140     xhat_error[2] = np.mod(xhat_error[2] + np.pi, 2 * np.pi) - np.pi
141
142     eps_k = np.zeros((1, prob.N_nodes))
143     for k in range(2, prob.N_nodes - 1):
144         eps_k_runs[i, k] = NEES(xhat_error[:, k], Phat_k[:, :, k])
145
146     print(i)
147
148 nu_k_avg = np.average(nu_k_runs, 0)
149 eps_k_avg = np.average(eps_k_runs, 0);
150
151 eps_bounds = averaged_chi_squared_consistency_bands(N_runs, 3)
152 nu_bounds = averaged_chi_squared_consistency_bands(N_runs, 2)
153
154 plt.subplot(1, 2, 1)
155 plt.plot(prob.t_k, eps_k_avg, color = "red")
156 plt.title("Time Averaged NEES")
157 plt.hlines(eps_bounds, 0, prob.T, linestyle = "--", label="95% bounds")
158 plt.grid()
159 plt.legend()
160
161 plt.subplot(1, 2, 2)
162 plt.plot(t_k_GPS, nu_k_avg, color = "red")
163 plt.title("Time Averaged NIS")
164 plt.hlines(nu_bounds, 0, prob.T, linestyle = "--", label="95% bounds")
165 plt.grid()
166 plt.legend()
167
168 plt.savefig("./HW4/Parts/Q2/AAE590LGM_HW4_Q2e_mc.png")
169 plt.show()

```